Pandas Quick Reference

Beginner-Friendly Cheat Sheet from Class Notes

Based on actual class notebooks | Corrected & organized for exam revision

1. Introduction to Pandas

Pandas is a Python library for working with tabular data. Two main objects:

• DataFrame – 2D table (rows + columns)

• Series – single column or single row (1D)

```
import pandas as pd

df = pd.read_csv('student_data.csv')
print(type(df))          # <class 'pandas.core.frame.DataFrame'>

student_id = df['StudentID']
print(type(student_id))  # <class 'pandas.core.series.Series'>
```

2. Loading Data from Different Files

Pandas can read many file formats into a DataFrame.

```
# CSV
csv_data = pd.read_csv('student_data.csv')

# Excel
excel_file = pd.read_excel('phitron_student_marks.xlsx')

# Parquet
parquet_file = pd.read_parquet('students.parquet')

# JSON
json_file = pd.read_json('data.json')
```

Tip: Always check the type: print(type(df)) → should be DataFrame.

3. Basic DataFrame Methods

3.1 head() and tail()

Show the first or last few rows.

```
df.head(4)   # first 4 rows (default is 5)
df.tail()    # last 5 rows
```

3.2 columns, index, info(), sample(), describe()

```
df.columns          # column names
df.index            # row labels (usually 0, 1, 2, ...)
df.info()           # column names, non-null counts, dtypes, memory
df.sample(10)       # 10 random rows
df.describe()       # statistical summary of numeric columns only
```

Note: *describe() shows count, mean, std, min, 25%, 50%, 75%, max for numeric columns.*

4. Creating DataFrames from Python Data

4.1 From a list of lists

```
my_list = [['Alice', 25], ['Bob', 30], ['Charlie', 28]]
list_df = pd.DataFrame(my_list, columns=['Name', 'Age'], index=[1, 2, 3])
list_df
```

4.2 From a tuple of tuples

```
my_tuple = (('Alice', 25), ('Bob', 30), ('Charlie', 28))
tuple_df = pd.DataFrame(my_tuple, columns=['Name', 'Age'])
tuple_df
```

4.3 From a dictionary

```
my_dict = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 29, 28]
}
dict_df = pd.DataFrame(my_dict)
dict_df
```

4.4 From a list of dictionaries

```
my_data = [
    {'Name': 'Alice', 'Age': 25, 'City': 'New York'},
    {'Name': 'Bob', 'Age': 30, 'City': 'Paris'},
    {'Name': 'Charlie', 'Age': 28}
]
data_df = pd.DataFrame(my_data)
data_df
```

Note: *Missing keys become NaN automatically.*

5. Accessing Values (loc and column selection)

Use square brackets or .loc for selecting rows and columns.

```
# Single column → Series
df['FullName']

# Single row by label
df.loc[0]

# Multiple specific rows
df.loc[[2, 3, 19]]

# Row range (inclusive)
df.loc[3:7]

# Single column with loc
df.loc[:, 'Python Marks']

# Multiple columns
df.loc[:, ['Python Marks', 'Algorithm Marks']]

# Rows + one column
df.loc[3:7, 'CompletionStatus']
```

Tip: loc uses labels (index names). iloc uses integer positions (0, 1, 2, ...).

6. Changing Index and Column Names

6.1 set_index()

```
# Creates a new DataFrame with StudentID as index
df_index = df.set_index('StudentID')

# To change original permanently:
# df.set_index('StudentID', inplace=True)
```

6.2 iloc (integer location)

After changing the index, use iloc for position-based selection.

```
df_index.iloc[:, 0:5]   # all rows, first 5 columns
```

6.3 rename columns

```
df.rename(
    columns={'FullName': 'Full Name', 'Algorithm Marks': 'Algo Marks'},
    inplace=True
)
```

7. Changing Values, Dropping Rows/Columns

7.1 Drop rows or columns

```
# Drop a row by index label
df.drop(0, inplace=True)

# Drop a column (axis=1 means columns)
df.drop('Instructor', axis=1, inplace=True)
```

7.2 Change cell values

```
df.loc[1, 'Python Marks'] = 90
df.loc[1, 'CompletionStatus'] = 'Completed'

# Update a range of cells
df.loc[1:3, 'Python Marks'] += 2
```

8. Iterating Over a DataFrame

```
# iterrows() → (index, Series of the row)
for i, series in df.iterrows():
    print(f"{i}:\n{series}")

# itertuples() → namedtuple (faster)
for row in df.itertuples(index=False):
    print(row)
```

Tip: Prefer vectorized operations over loops when possible. Loops are slower on large data.

9. Sorting

```
# Ascending (default)
copy = df.sort_values('Data Structure Marks')

# Descending
```

```
copy = df.sort_values('Data Structure Marks', ascending=False)

# Multiple columns (both descending)
copy = df.sort_values(['Data Structure Marks', 'Python Marks'], ascending=False)

# Different directions: DS descending, Python ascending
copy = df.sort_values(
    ['Data Structure Marks', 'Python Marks'],
    ascending=[False, True]
)
```

Note: *ascending=False is clearer than ascending=0. Both work, but False is more readable.*

10. Filtering Data with Conditions

```
# Simple condition
not_started = df.loc[df['CompletionStatus'] == 'Not Started']
completed = df.loc[df['CompletionStatus'] == 'Completed']

# Multiple conditions (use & for AND, | for OR; put each condition in parentheses)
completed_ds90 = df.loc[
    (df['CompletionStatus'] == 'Completed') &
    (df['Data Structure Marks'] >= 90)
]
```

11. String Filtering with str.contains()

Useful for searching text inside string columns. Supports regular expressions.

```
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve', 'Frank', 'Grace', 'Hannah', 'Sakib'],
    'City': ['New York', 'Los Angeles', 'Newark', 'Boston', 'New Delhi',
             'Chicago', 'New Orleans', 'Houston', 'H Los Ang'],
    'Department': ['HR', 'IT', 'Finance', 'IT', 'HR', 'Marketing', 'Finance', 'HR', 'HR'],
    'Salary': [50000, 60000, 55000, 70000, 52000, 58000, 62000, 51000, 70000]
}
df = pd.DataFrame(data)

# Contains 'New' (case-sensitive)
df.loc[df['City'].str.contains('New')]

# Case-insensitive
df.loc[df['City'].str.contains('new', case=False)]

# Starts with 'Los' (regex)
df.loc[df['City'].str.contains(r'^Los')]

# Ends with 'rk'
df.loc[df['City'].str.contains(r'rk$')]

# Name starts with a vowel
df.loc[df['Name'].str.contains(r'^[AEIOU]')]

# Contains 'New' OR 'Los'
df.loc[df['City'].str.contains(r'New|Los')]
```

12. Adding New Columns

```
df = pd.read_csv('student_data.csv')

# Constant value
df['Country'] = 'Bangladesh'

# Arithmetic from other columns
df['Total Marks'] = (df['Data Structure Marks'] +
                     df['Python Marks'] +
                     df['Algorithm Marks'])

# Conditional with np.where
import numpy as np
df['A+ in DS'] = np.where(df['Data Structure Marks'] > 90, 'A+', 'A')

# Boolean column
df['Passed in DS'] = df['Data Structure Marks'] > 70

# Extract first name from FullName
df['First Name'] = df['FullName'].str.split(' ').str[0]
```

Tip: You can also write a DataFrame to CSV: df.to_csv('new_data.csv')

13. unique() and nunique()

```
df['Name'].unique()          # array of unique values (includes NaN)
len(df['Name'].unique())

df['Data Structure Marks'].nunique()  # count of unique values (excludes NaN by default)
df.nunique()                          # unique count for every column
```

14. Checking for Null / Missing Values

```
df.isnull()                     # DataFrame of True/False
df['Data Structure Marks'].isnull()   # Series of True/False
df['Data Structure Marks'].notnull()  # opposite of isnull()
df['Data Structure Marks'].hasnans    # True if any NaN exists in the Series
```

15. Handling Duplicate Rows

```
data = {
    'Name': ['Alice', 'Bob', 'Charlie', 'Alice', 'David', 'Bob'],
    'City': ['New York', 'London', 'Paris', 'New York', 'Tokyo', 'London'],
    'Score': [85, 90, 78, 70, 95, 90]
}
df = pd.DataFrame(data)

df.duplicated()              # True for duplicate rows
df.duplicated().sum()        # how many duplicates

df.drop_duplicates()                    # remove full-row duplicates
df.drop_duplicates(inplace=True)        # permanent

df.drop_duplicates(subset=['Name'])             # based on Name only
df.drop_duplicates(subset=['Name', 'City'])
```

```
df.drop_duplicates(subset=['Name'], keep='last') # keep last occurrence
df.drop_duplicates(subset=['Name'], keep='first')# keep first (default)
```

16. Handling Null Values

16.1 Dropping nulls

```
df.dropna()                              # drop any row that has at least one NaN
df.dropna(how='all')                     # drop only if ALL values in the row are NaN
df.dropna(subset=['Python Marks'])       # drop if Python Marks is NaN
df.dropna(subset=['Python Marks', 'Algorithm Marks'])  # either is NaN
```

16.2 Filling nulls

```
df.fillna(0)                             # fill every NaN with 0
df['FullName'].fillna('unknown')         # fill only one column

# Fill with the mean of that column
df['Python Marks'].fillna(df['Python Marks'].mean(), inplace=True)
```

Note: *fillna(0) on a name column is usually not useful. Prefer a meaningful value or the column mean/median.*

17. Statistical Functions

```
df['Data Structure Marks'].sum()
df['Data Structure Marks'].min()
df['Data Structure Marks'].max()
df['Data Structure Marks'].mean()      # average
df['Data Structure Marks'].median()
df['Data Structure Marks'].mode()      # most frequent value(s)
df['Data Structure Marks'].std()       # standard deviation

# Correlation between two columns
df[['Data Structure Marks', 'Python Marks']].corr()

# Sum across columns (column-wise)
df[['Data Structure Marks', 'Python Marks']].sum()

# Sum across rows (row-wise)
df[['Data Structure Marks', 'Python Marks']].sum(axis=1)

# Create Total Marks with iloc
df['Total Marks'] = df.iloc[:, 2:5].sum(axis=1)

df.describe()   # full statistical summary
```

18. apply() – Custom Functions

apply() lets you run a function on every value of a Series or every row/column of a DataFrame.

```
# Min-max scaling to 0–1 range
mn = df['Total Marks'].min()
mx = df['Total Marks'].max()
df['Scaled Marks'] = df['Total Marks'].apply(lambda x: (x - mn) / (mx - mn))

# Custom grading function
def grading_system(marks):
    if marks >= 260:
```

```
        return 'A+'
    elif marks >= 250:
        return 'A'
    else:
        return 'A-'

df['Grade'] = df['Total Marks'].apply(grading_system)

# Apply a function to each row (axis=1)
def marking_system(row):
    a = row['Data Structure Marks'] * 2
    b = row['Python Marks'] * 3
    c = row['Algorithm Marks'] * 4
    return a + b + c

df['Exceptional Marks'] = df.apply(marking_system, axis=1)
```

19. DateTime and Timedelta

```
df = pd.read_csv('student_completed_data.csv')

# Convert string column to datetime
df['EnrollmentDate'] = pd.to_datetime(df['EnrollmentDate'])
df['FinishedDate'] = pd.to_datetime(df['FinishedDate'])

# Extract year and day
df['Enrollment Year'] = df['EnrollmentDate'].dt.year
df['Enrollment Day'] = df['EnrollmentDate'].dt.day

# Time difference (Timedelta)
df['Total time taken to finish'] = df['FinishedDate'] - df['EnrollmentDate']
```

Tip: After conversion the dtype becomes datetime64[ns]. You can then use .dt.year, .dt.month, .dt.day, etc.

20. GroupBy

Group rows that share the same value in a column, then aggregate.

```
instructor_group = df.groupby('Instructor')

# Loop over groups
for instructor, df_ins in instructor_group:
    print(instructor)
    print(df_ins)

# Aggregations on each group
instructor_group.min()
instructor_group.describe()

# Mean of Total Marks per instructor
mean_marks = instructor_group['Total Marks'].mean()

# Fill NaN in Total Marks with the group mean of that instructor
df['Total Marks'] = df['Total Marks'].fillna(
    df['Instructor'].map(mean_marks)
)
```

Note: *map() replaces each Instructor name with its corresponding mean value, so fillna uses the correct group average.*

21. Quick Syntax Cheat Sheet

```
pd.read_csv / read_excel / read_json / read_parquet
df.head() / df.tail() / df.info() / df.describe() / df.sample(n)
df.columns / df.index / df.shape
df['col'] / df.loc[rows, cols] / df.iloc[row_pos, col_pos]
df.set_index('col') / df.rename(columns={...})
df.drop(index) / df.drop('col', axis=1)
df.sort_values('col', ascending=False)
df.loc[condition] / df.loc[(cond1) & (cond2)]
df['col'].str.contains('text', case=False)
df['new'] = value / expression / np.where(...)
df['col'].unique() / .nunique()
df.isnull() / .notnull() / .hasnans
df.duplicated() / df.drop_duplicates(subset=..., keep=...)
df.dropna(subset=...) / df.fillna(value)
df['col'].sum/mean/min/max/median/mode/std
df.apply(func) / df.apply(func, axis=1)
pd.to_datetime(df['col']) / .dt.year / .dt.day
df.groupby('col').mean() / .min() / .describe()
```

End of Quick Reference

All examples are corrected for syntax and are ready to run. Practice by typing them yourself.